第 12 章練習解答

1

若 tree，由連通與無 cycle，任何 spanning subgraph 有至少 n−1 edges，而 acyclic graph 有至多 n−1 edges，且 connected 迫等號（可用 induction 刪 leaf）。

反之若有 $n-1$ edges 且無 cycle，forest 的 components 數是 $c = n - e = 1$，所以 connected，故為 tree。

2

Full binary tree 滿足 $L = I + 1$，所以 $I = 30$。Edges 數是 $I + L - 1 = 60$。最小 height 方面，height 為 h 的 full binary tree 至多 2^h 個 leaves；又 $2^4 = 16 < 31$、$2^5 = 32$，所以最少 $h = 5$（root depth 設為 0）。可達性可由高度 5 的 perfect binary tree（32 leaves），把最底層一對 sibling leaves 收縮成其 parent leaf，得到 31 leaves；它仍是 full binary tree，高度仍為 5。

3

$6! = 720$，binary tree height H 的 leaves 至多 2^H，必須 $2^H \geq 720$。$2^9 = 512 < 720$，$2^{10} = 1024$，故 $H \geq 10$。$\lceil \log_2 720 \rceil = 10$。

Merge sort 先拆為 [5,1,4] 與 [2,6,3]。左半再拆 [5]、[1,4]：[1] 與 [4] 合併用 1 次，[5] 與 [1,4] 合併依序比 5 對 1、5 對 4，用 2 次，所以左半共 3 次，結果 [1,4,5]。右半拆 [2]、[6,3]：[6] 與 [3] 用 1 次，[2] 與 [3,6] 只比 2 對 3，用 1 次，所以右半共 2 次，結果 [2,3,6]。

最後合併的五次比較是 (1,2)、(4,2)、(4,3)、(4,6)、(5,6)，輸出前五項 1、2、3、4、5，再附上 6。總共 3+2+5=10 次。這只是一個輸入的成本，不能證最壞也是 10。此種 split 的最壞上界為 M(6)=2M(3)+5=2・3+5=11，且例如 [4,2,6,3,1,5] 可使兩半各用 3 次、最後交錯用 5 次，達到 11。因此本次剛好等於資訊下界，不代表此演算法整體達到六鍵最優最壞比較數。

4

合併 2+3=5；權重集合 5(new),5,7,11；合併 5+5=10；剩 7,10,11；合併 7+10=17；最後 17+11=28。Merge total=5+10+17+28=60。

按合併樹可取 code：11 為 `0`；7 為 `10`；5 為 `110`；2 為 `1110`；3 為 `1111`。Weighted cost 為

$$11 \cdot 1 + 7 \cdot 2 + 5 \cdot 3 + 2 \cdot 4 + 3 \cdot 4 = 60,$$

平均 $60/(2+3+5+7+11) = 60/28 = 15/7$。左右標記互換仍是同一組最優長度。

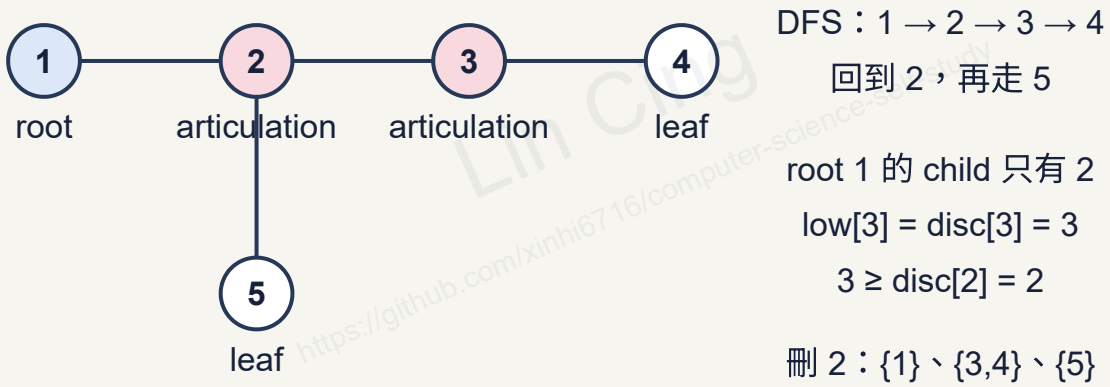
第 12 章練習解答 （續）

5

Connected 保證至少一條 path。若有兩條 distinct simple paths，取它們第一次分開的共同 prefix 與第一次重新接回的共同 suffix；兩段內部不相交的中間 paths 會構成 simple cycle，與 tree 無 cycle 矛盾。不能只說「最短非零 closed walk 必為 cycle」，因來回同一條 edge 也可形成長度 2 的 closed walk。因此 paths 至多一條，連通性又保證至少一條。

6

取 path $1-2-3-4$，再加上 edge $2-5$。若 DFS root 是 1，DFS tree 先沿 $1-2$ 前進；接著 2 可連到 3 與 5，而 root 只有 child 2。非 root 的 2 是 articulation：刪除 2 後，vertex 1 與 vertices 3、4、5 分離。以 root 1 執行 DFS 時，$low[3]$ 不可能經 back edge 回到 root，且 $disc[2] < disc[3]$、$low[3] \geq disc[2]$，所以對 child 3 的條件成立。刪除 vertex 3 也會使 4 與其餘部分分離；同一 low-link 判準會指出 3 也是 articulation。最後的 articulation set 是 $\{2, 3\}$。



7

固定 IDs 為 e_1=AB、e_2=BC、e_3=CA、e_4=CD、e_5=DE、e_6=EF、e_7=FD、e_8=DG。由 A 開始的發現順序 A、B、C、D、E、F、G，disc=1、2、3、4、5、6、7，parent 依序為無、A、B、C、D、E、D。

push 順序先為 1、2、3、4、5、6、7。回邊 3 把 C 的 low 改為 1；回邊 7 把 F 的 low 改為 4，返回 E 後 low[E]=4。E 返回 D 時 low[E]=disc[D]=4，完整 pop 7、6、5，得到 DEF 三角形。再 push 8，G 返回 D 時 low[G]=7≥4，pop 8，得到橋 DG。D 返回 C 時 low[D]=4≥3，pop 4，得到橋 CD。C 返回 B 時 low[C]=1<2，不 pop；B 返回 A 時 low[B]=1≥1，pop 3、2、1，得到 ABC 三角形。

第 12 章練習解答 （續）

返回後 low 依 A 至 G 為 1、1、1、4、4、4、7。非根 C 的 child D 滿足 4≥3；非根 D 的 children E、G 都滿足 low≥4，因此 cut set 為 {C,D}。橋用嚴格大於，只有 e_4、e_8。所有八條邊各歸入一個 block，兩個三角形完整保留。

改以 D 為根，按字母序先由 C 走訪 A、B，返回後再由 E 走 F，最後 G，因此根有三個 DFS children C、E、G。刪 D 後恰分成 ABC、EF、G 三個分量，根確為 articulation。C 的通往 AB 子樹無法繞回 D，仍為 articulation。disc、low 及 pop 次序可以改變；圖本身的 cut set {C,D}、bridges {CD,DG} 與四個 blocks 不改變。

8

(1,2,2,3) 的 Kraft 和為 1/2+1/4+1/4+1/8=9/8>1，故不存在。不能只嘗試某幾組字串失敗就宣告不可能，因為 Kraft 是對所有 prefix code 的必要條件。

(2,2,3,3,3,3) 的和為 2/4+4/8=1，可取 `00,01,100,101,110,111`。前兩個都以 0 開頭，後四個以 1 開頭；各自同長且互異，所以沒有一個 codeword 是另一個的 prefix。

單一符號且已知來源符號數時，可以用空字串 ε、平均長度 0，接收端以次數還原。若每個 codeword 必非空，最小長度至少 1，可用 `0` 達成。規格不同，最優目標值不同；不能一面要求非空一面引用空字串最優值。

9

m=3、I=13，所以 n=mI+1=40，L=(m−1)I+1=27，e=n−1=39。由高度 h 的三元樹葉數至多 3^h，27≤3^h 給 h≥3。此界可達：取 height 3 的 perfect ternary tree，內部點數 1+3+9=13、葉數 27，恰滿足要求。它不是只符合計數上界的假想樹，逐層節點為 1、3、9、27，邊數為 3+9+27=39。

10

至少兩個符號且權重全正時，最優 prefix tree 不含單一 child 的內部點，否則可收縮並嚴格降低後代成本。因此三葉最優樹必為 full binary，形狀只能是根的一個 child 為葉，另一個為內部點並帶兩葉，深度集合為 (1,2,2)。

若深度 1 的符號權重為 w，總成本是 2(x+y+z)−w。要最小就必須選最大權重 z，得到 W=2x+2y+z。這也由 Huffman 先合併 x+y、再合併 x+y+z 得到相同值。可取 z:`0`、x:`10`、y:`11`。

若 z 與某較小的 w 交換到深度 1，成本改變為 w・2+z・1−(w・1+z・2)=w−z≤0，所以讓 z 留在深度 2 不可能比把它移到深度 1 更佳；相等權重可能有多個最優解。x=2、y=5、z=9 時合併值 7、16，W=23，平均長度 23/16=1.4375 bits。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 12 章練習解答 （續）

11

若刪樹邊 uv 後仍可從 u 走到 v，這條替代 simple path 加上 uv 就形成 cycle，與樹無環矛盾，所以每條邊都是橋。握手引理與 e=n−1 給

$$\sum_{v}(\deg(v)-2)=2e-2n=2(n-1)-2n=-2.$$

當 n≥2 沒有 degree 0 點。設葉數 L，degree 2 點貢獻 0，三個 degree 3 點共貢獻 3，一個 degree 5 點貢獻 3，葉各貢獻 −1。故 −L+3+3=−2，得到 L=8。

n=1 的唯一點 degree 0，對上式貢獻 −2，所以上式仍成立；但它不是 degree 1 的葉。因此若直接把所有負貢獻都當 −L，會漏掉 degree 0 的 −2。Rooted-tree 的葉定義則以無 children 為準，單點樹有一個 rooted leaf，這與 degree 1 定義不同。

12

令 disc[u]=1，沿 e_1 到 v，disc[v]=2。只跳過 parent edge ID e_1，另一條 e_2 仍被掃到，它連回祖先 u，因此 low[v]=min(2,disc[u])=1。返回 u 時 low[v]=disc[u]，不符合橋所需的嚴格大於，所以 e_1 不是橋。實際刪去 e_1 後仍有 e_2 連接兩點，直接驗證此答案。

若把所有指向 parent vertex u 的邊都跳過，e_2 也被錯略過，low[v] 留為 2，便錯把 e_1 判成橋。Edge stack 的正確順序是先推 e_1、再推回邊 e_2，返回時 low[v]≥disc[u]，一次 pop e_2、e_1，形成包含兩條平行邊的 block。根 u 只有一個 child，因此不是 articulation。

13

六條邊各自在三角形上，所以沒有橋。刪 C 後分成 {A,B}、{D,E} 兩個分量，故 C 是割點。刪 A 或 B，另一點仍接到 C；刪 D 或 E 同理，因此沒有其他割點。兩個 maximal、連通且無割點的子圖為 ABC、CDE。Block-cut tree 為 B_1–C–B_2，其中 B_1=ABC、B_2=CDE。原圖沒有橋可刪，刪橋後仍保留整張連通圖，並不會分成這兩個 vertex blocks。

加入 BD 後，刪 C 留下連通路徑 A–B–D–E。刪其他點時 C 仍在：刪 A 留下 B–C 與三角形 CDE；刪 B 留下 A–C 與三角形 CDE；刪 D 留下三角形 ABC 與 E–C；刪 E 留下三角形 ABC 與 D–C。每種情形都連通，所以不再有割點，整張圖就是一個 vertex block。新邊 BD 在環 B–C–D–B 上，原有邊仍在各自三角形上，因此仍沒有橋。新的 block-cut tree 只有一個 block 節點，沒有 cut 節點。

參考資料與署名

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study